LISTAS Y COMBOBOX EN SWING

JList, JComboBox y el patrón Modelo-Vista-Controlador

MATERIAL DE CLASE · DESARROLLO DE SISTEMAS ORIENTADO A OBJETOS

Esteban Santini · 2026

AGENDA DE LA CLASE

01 · JLIST: LISTAS DE SELECCIÓN

Qué es un JList, cómo se relaciona con su ListModel y la diferencia entre datos fijos y datos dinámicos.

02 · JCOMBOBOX: LISTAS DESPLEGABLES

El componente para elegir una única opción entre varias, su ComboBoxModel y sus eventos.

03 · SELECCIÓN DEPENDIENTE

El patrón donde un combo decide qué mostrar y una lista lo muestra, actualizando su modelo.

04 · EJEMPLO MVC COMPLETO

Catálogo de productos por categoría: modelo, vista y controlador trabajando juntos.

JLIST: LISTAS DE SELECCIÓN

javax.swing.JList — mostrar una colección de opciones

- Muestra una colección de elementos en forma de lista vertical, de la que el usuario puede elegir uno o varios.
- Es un componente parametrizado: JList<String> asegura en tiempo de compilación qué tipo de datos contiene.
- Por defecto permite elegir un solo elemento (SINGLE_SELECTION); se puede configurar para selección múltiple.
- Casi siempre se usa envuelto en un JScrollPane, para poder desplazarse cuando hay más elementos que espacio visible.
- getSelectedValue() devuelve el elemento elegido; getSelectedIndex() devuelve su posición dentro de la lista.

COMPONENTES RELACIONADOS

JList trabaja siempre junto con:

```
ListModel          →  el modelo de datos
JScrollPane        →  para poder scrollear
ListSelectionListener → escuchar cambios
import javax.swing.*;
import javax.swing.event.*;
```

JLIST Y SU MODELO DE DATOS

DATOS FIJOS (ARRAY)

- new JList<>(array) construye internamente un ListModel de solo lectura a partir del array.
- Sirve para catálogos que no van a cambiar mientras la ventana esté abierta.
- Para modificarlo hay que crear una lista nueva y reemplazar el componente: poco práctico.
- Ejemplo: JList<String> lista = new JList<>(new String[]{"A", "B"});

DATOS DINÁMICOS (DefaultListModel)

- DefaultListModel<String> es un modelo mutable: addElement(), removeElement(), clear().
- El JList se construye una sola vez, apuntando siempre al mismo modelo.
- Cualquier cambio en el modelo repinta la lista automáticamente: no hay que tocar el JList.
- Es la opción correcta cuando el contenido depende de una acción del usuario, como elegir una categoría.

JLIST: CÓDIGO BÁSICO

- Se puede construir con datos fijos (un array) o con un modelo dinámico.
- setVisibleRowCount() sugiere cuántas filas mostrar antes de necesitar scroll.
- Como todo componente Swing, conviene envolverlo en un JScrollPane antes de agregarlo al contenedor.
- getSelectedValue() devuelve directamente el objeto elegido, ya tipado como String.

```
// Instanciar (con datos fijos)
String[] frutas = {"Manzana", "Banana", "Naranja"};
JList<String> lista = new JList<>(frutas);

// Configurar atributos
lista.setSelectionMode(
    ListSelectionModel.SINGLE_SELECTION);
lista.setVisibleRowCount(5);

// Componer (con scroll)
ventana.add(new JScrollPane(lista));

// Leer la selección
String elegido = lista.getSelectedValue();
```

JCOMBOBOX: LISTAS DESPLEGABLES

javax.swing.JComboBox — elegir una opción entre varias

- Combina un campo visible con una lista desplegable: se ve una opción a la vez y se despliega el resto al hacer clic.
- También es parametrizado: JComboBox<String> garantiza el tipo de los elementos que contiene.
- Puede ser de solo selección (por defecto) o editable, permitiendo escribir un valor con setEditable(true).
- getSelectedItem() devuelve la opción elegida como Object: conviene castearlo o usar el tipo parametrizado.
- setSelectedIndex(0) o setSelectedItem(valor) permiten fijar la selección desde el código.

COMPONENTES RELACIONADOS

JComboBox trabaja siempre junto con:

```
ComboBoxModel    →  el modelo de datos
ActionListener   →  escuchar cambios
ItemListener     →  alternativa específica
import javax.swing.*;
import java.awt.event.*;
```

JCOMBOBOX Y SU MODELO DE DATOS

ITEMS FIJOS EN EL CONSTRUCTOR

- new JComboBox<>(array) arma un ComboBoxModel de solo lectura con esos valores.
- addItem() todavía funciona para sumar opciones sueltas al modelo por defecto.
- No está pensado para reconstruir todo el contenido con frecuencia.

DefaultComboBoxModel

- DefaultComboBoxModel<String> es el modelo mutable detrás de cada JComboBox por defecto.
- removeAllElements() + addElement() en bucle reemplaza prolijamente todas las opciones.
- setModel() permite reemplazar el modelo completo del combo en tiempo de ejecución.

JCOMBOBOX: CÓDIGO BÁSICO

- addItem() agrega opciones en el orden en que se van a mostrar.
- setSelectedIndex(0) fija una selección inicial: sin esto, arranca vacío.
- El combo dispara un evento cada vez que cambia la opción elegida.
- getSelectedItem() devuelve Object: hace falta castear al tipo real.

```
// Instanciar
JComboBox<String> combo = new JComboBox<>();

// Configurar atributos
combo.addItem("Frutas");
combo.addItem("Bebidas");
combo.addItem("Lácteos");
combo.setSelectedIndex(0);

// Componer
ventana.add(combo);

// Leer la selección
String elegido =
    (String) combo.getSelectedItem();
```

EVENTOS: JLIST vs JCOMBOBOX

JLIST

- Se escucha con addListSelectionListener(ListSelectionListener).
- El método a implementar es valueChanged(ListSelectionEvent e).
- e.getValueIsAdjusting() es true mientras el usuario todavía está arrastrando la selección: conviene ignorar esos eventos intermedios.
- Se dispara con cada cambio de selección, incluso al deseleccionar.

JCOMBOBOX

- Se escucha con addActionListener(ActionListener), igual que un JButton.
- El método a implementar es actionPerformed(ActionEvent e).
- Se dispara cuando el usuario confirma una nueva selección.
- getSelectedItem() dentro del listener siempre devuelve la opción vigente en ese momento.

Ambos forman parte del mismo mecanismo Observer que ya vimos con ActionListener en JButton: el componente avisa, el listener reacciona.

COMBINARLOS: SELECCIÓN DEPENDIENTE

El patrón combo → lista

- Es un patrón muy común en interfaces de escritorio: un control decide "qué mostrar" y otro lo muestra.
- Se registra un ActionListener sobre el JComboBox, no sobre el JList: la lista no dispara el cambio, lo recibe.
- La clave es no reemplazar el JList por uno nuevo: se actualiza el DefaultListModel que ya tiene asignado.
- El listener hace siempre lo mismo: lee la selección del combo, consulta el modelo de datos y reconstruye la lista con clear() + addElement().

PASOS DEL PATRÓN

1
El usuario elige una categoría en el combo.

2
Se dispara el ActionListener registrado.

3
El controlador limpia y repuebla el modelo de la lista.

EL EJEMPLO: CATÁLOGO POR CATEGORÍA

Un JComboBox que decide, un JList que muestra

- El combo lista las categorías disponibles: Frutas, Bebidas, Lácteos.
- La lista muestra los productos de la categoría actualmente seleccionada en el combo.
- Al cambiar de categoría, la lista se actualiza sola: nunca hay productos de dos categorías mezclados.
- Lo vamos a construir con tres clases: CatalogoModelo, CatalogoVista y CatalogoControlador.

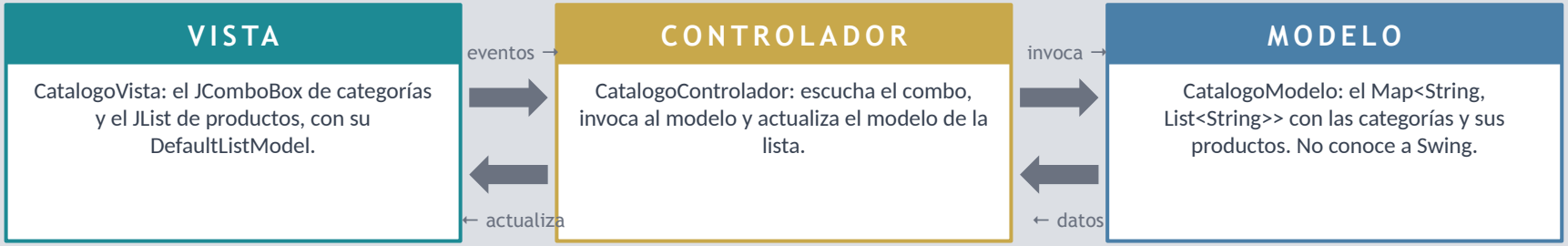
ESTRUCTURA DE DATOS

El modelo guarda esta relación:

```
Frutas    →  Manzana, Banana...
Bebidas   →  Agua, Jugo...
Lácteos   →  Leche, Yogur...
Map<String, List<String>>
clave = categoría elegida
```

ARQUITECTURA MVC APLICADA

Aplicamos el mismo esquema Modelo–Vista–Controlador que ya conocemos, ahora con JComboBox y JList como parte de la vista.



- Regla de oro (la misma de siempre): el modelo nunca importa javax.swing.
- El controlador es el único que conecta el combo con la lista: ninguno de los dos se conoce directamente.
- Vamos a construir las tres clases del "Catálogo" en las próximas diapositivas.

MVC EN SWING — 1) EL MODELO

- Un simple Map<String, List<String>> guarda qué productos pertenecen a cada categoría.
- No tiene ni un import de javax.swing: es una clase de datos común y corriente.
- obtenerCategorias() expone las claves; obtenerProductos() expone los valores de una categoría.
- Al no depender de la interfaz, se puede probar con JUnit sin abrir ninguna ventana.

```
// MODELO: datos y lógica, sin Swing
public class CatalogoModelo {

    private final Map<String, List<String>> datos =
        new LinkedHashMap<>();

    public CatalogoModelo() {
        datos.put("Frutas", List.of(
            "Manzana", "Banana", "Naranja", "Pera"));
        datos.put("Bebidas", List.of(
            "Agua", "Jugo", "Gaseosa"));
        datos.put("Lácteos", List.of(
            "Leche", "Yogur", "Queso"));
    }

    public List<String> obtenerCategorias() {
        return new ArrayList<>(datos.keySet());
    }

    public List<String> obtenerProductos(String cat) {
        return datos.getOrDefault(cat, List.of());
    }
}
```

MVC EN SWING — 2) LA VISTA

- Extiende JFrame y expone dos componentes clave como atributos: el combo y el modelo de la lista.
- El JList se crea vacío, apuntando a un DefaultListModel propio: el controlador lo llena después.
- BorderLayout ubica el combo arriba (NORTH) y la lista, con scroll, en el centro (CENTER).
- No decide qué productos mostrar: solo expone getComboCategorias() y getModeloLista() para que el controlador actúe.

```
// VISTA: solo interfaz, sin lógica de negocio
public class CatalogoVista extends JFrame {

    private final JComboBox<String> comboCategorias =
        new JComboBox<>();
    private final DefaultListModel<String> modeloLista =
        new DefaultListModel<>();
    private final JList<String> listaProductos =
        new JList<>(modeloLista);

    public CatalogoVista() {
        super("Catálogo por categoría");
        setLayout(new BorderLayout(10, 10));
        add(comboCategorias, BorderLayout.NORTH);
        add(new JScrollPane(listaProductos),
            BorderLayout.CENTER);
        setSize(320, 280);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
    }

    public JComboBox<String> getComboCategorias() {
        return comboCategorias;
    }

    public DefaultListModel<String> getModeloLista() {
        return modeloLista;
    }
}
```

MVC EN SWING — 3) EL CONTROLADOR

```
// CONTROLADOR: conecta la vista con el modelo
public class CatalogoControlador {

    private final CatalogoModelo modelo;
    private final CatalogoVista vista;

    public CatalogoControlador(CatalogoModelo modelo, CatalogoVista vista)
{
        this.modelo = modelo;
        this.vista = vista;
        modelo.obtenerCategorias().forEach(
            vista.getComboCategorias()::addItem);
        vista.getComboCategorias()
             .addActionListener(e -> actualizarLista());
        actualizarLista();
    }

    private void actualizarLista() {
        String cat = (String) vista.getComboCategorias()
                                   .getSelectedItem();
        DefaultListModel<String> modeloLista = vista.getModeloLista();
        modeloLista.clear();
        modelo.obtenerProductos(cat).forEach(modeloLista::addElement);
    }
}
```

- Antes de escuchar eventos, carga las categorías del modelo dentro del combo.
- El único listener registrado es el del combo: un ActionListener con una lambda.
- actualizarLista() es el corazón del patrón: lee la categoría elegida, limpia el modelo de la lista y lo vuelve a llenar.
- El main solo arma las tres piezas y muestra la ventana, dentro de SwingUtilities.invokeLater().

EL HILO DE EVENTOS: INVOKELATER

javax.swing.SwingUtilities — ejecutar código en el EDT

- Swing no es thread-safe: crear o modificar componentes visuales solo es seguro dentro de un único hilo especial, el Event Dispatch Thread (EDT).
- El método main() corre en el hilo principal de la JVM, no en el EDT: por eso nunca hay que instanciar una ventana Swing directamente ahí.
- SwingUtilities.invokeLater(Runnable) encola la tarea para que se ejecute en el EDT tan pronto como quede libre, sin bloquear al hilo que la invoca.
- Es asincrónico: el hilo que llamó a invokeLater() sigue su ejecución de inmediato, sin esperar a que la tarea termine de correr en el EDT.
- Por convención, todo main() de una aplicación Swing arranca así: invokeLater(() -> { crear vista y controlador }).

INVOKELATER vs INVOKEANDWAIT

Dos formas de mandar trabajo al EDT:

```
invokeLater      → asíncrono, no bloquea
invokeAndWait    → bloquea el hilo llamador
Fuera del EDT    → riesgo de bugs intermitentes
import javax.swing.SwingUtilities;
// nunca crear la vista fuera del EDT
```

RESULTADO EN EJECUCIÓN

```
// PUESTA EN MARCHA
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        CatalogoVista vista = new CatalogoVista();
        new CatalogoControlador(
            new CatalogoModelo(), vista);
        vista.setVisible(true);
    });
}
```

RESULTADO



Las tres clases ya están armadas: el main solo las instancia y las conecta.
Cada clic en el combo dispara actualizarLista(), que repuebla el JList a partir del modelo.

BUENAS PRÁCTICAS Y ERRORES COMUNES

BUENAS PRÁCTICAS

- Actualizar el modelo existente (clear() + addElement()) en vez de crear un JList nuevo cada vez.
- Parametrizar los componentes — JList<String>, JComboBox<String> — para evitar casteos innecesarios.
- Envolver el JList en un JScrollPane: sin scroll, las listas largas se cortan.
- Construir la interfaz dentro de SwingUtilities.invokeLater(), como cualquier aplicación Swing.

ERRORES COMUNES

- Reasignar el JList a un componente nuevo en cada evento: se pierde la selección y el listener queda huérfano.
- Olvidar el casteo (String) al leer getSelectedItem(), que devuelve Object salvo que el combo esté parametrizado.
- Registrar el listener del combo antes de cargar las categorías: dispara actualizaciones con el modelo todavía vacío.
- Mezclar lógica de negocio en la vista, por ejemplo filtrando productos directamente en el ActionListener.

Estos son los mismos errores que aparecen en cualquier componente Swing basado en un modelo: la solución siempre es actualizar el modelo, nunca el componente.

SELECCIÓN MÚLTIPLE EN JLIST

- Por defecto, un JList permite elegir un solo elemento a la vez (SINGLE_SELECTION).
- setSelectionMode() cambia el modo: SINGLE_INTERVAL_SELECTION permite un rango contiguo.
- MULTIPLE_INTERVAL_SELECTION habilita selección múltiple libre, con Ctrl/Shift + clic.
- getSelectedValuesList() devuelve todos los elementos elegidos, no solo el primero.

```
// Habilitar selección múltiple
lista.setSelectionMode(
    ListSelectionModel
        .MULTIPLE_INTERVAL_SELECTION);

// Leer todos los seleccionados
List<String> elegidos =
    lista.getSelectedValuesList();

for (String producto : elegidos) {
    System.out.println(producto);
}
```

RESUMEN Y BUENAS PRÁCTICAS

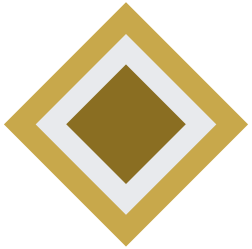
- JList y JComboBox ya separan datos de presentación internamente: ListModel y ComboBoxModel.
- Para contenido que cambia, usá DefaultListModel y DefaultComboBoxModel: nunca reconstruyas el componente.
- El patrón combo → lista se resuelve con un solo listener sobre el combo que actualiza el modelo de la lista.
- La arquitectura MVC que ya conocés se aplica igual: modelo sin Swing, vista sin lógica, controlador coordinando.
- El ejemplo completo (CatalogoModelo, CatalogoVista, CatalogoControlador) queda como referencia para la práctica.

SIDE QUEST

Práctica: agregá un tercer control al catálogo.

Un JComboBox de orden (alfabético / por cantidad de letras) que reordene la lista visible, sin tocar el modelo de datos original.

Documentación: docs.oracle.com → How to Use Lists → How to Use Combo Boxes



MUCHAS GRACIAS

"Una buena interfaz no muestra todo: muestra lo que corresponde a cada momento."

Preguntas y consultas: ahora, o por los canales de la materia.
En la próxima clase: práctica integradora de Swing + MVC.